HTML Cheat Sheet for Web Scraping

Python · requests · BeautifulSoup · CSS Selectors

Inspect → Understand → Select → Extract

HTML Cheat Sheet for Web Scraping with Python & BeautifulSoup

> **Purpose of this doc:** You're not learning HTML to build websites. You're learning it to *read* a page well enough to answer one question, over and over: *"How do I write BeautifulSoup code that reliably grabs this exact piece of data?"* Everything below is filtered through that lens.

---

0. The One Mental Model That Explains Everything

An HTML page is not really "code" — it's a **tree**.

```
<div class="product">
    <h2 class="title">Wireless Mouse</h2>
    <span class="price">$19.99</span>
    <a href="/product/123">View</a>
</div>
```

This is a tree:

```
div.product
├── h2.title      → "Wireless Mouse"
├── span.price    → "$19.99"
└── a[href]       → "/product/123"
```

Why this matters for scraping: BeautifulSoup doesn't "read" HTML like text — it builds this exact tree in memory and gives you methods to walk it (`find` , `find_all` , `.parent` , `.select` , etc.). Every scraping decision you make is really just answering: *"Where in this tree does my data live, and what's the shortest reliable path to it?"*

So your job when inspecting a page is always: 1. Find the data visually on the page. 2. Find its node in the tree (via DevTools). 3. Find a **pattern** that identifies that node (or its siblings, if there are many). 4. Translate that pattern into BeautifulSoup/CSS syntax.

That's the whole workflow. Let's build up the vocabulary to do it.

---

1. The Workflow You'll Repeat Every Time

```
Website
   ↓
Right-click → Inspect Element
   ↓
Find the HTML node behind the visible data
   ↓
Look at its tag, attributes (class/id), and its parents/siblings
   ↓
Find the pattern that repeats (if scraping a list) or that's unique (if scraping one item)
   ↓
Write a find() / find_all() / select() that captures exactly that pattern
   ↓
Extract text or attribute value
```

The rest of this cheat sheet is organized around these steps.

---

2. Anatomy of a Tag (Only What You Need)

```
<a class="product-link" href="/item/42" data-id="42">Buy Now</a>
```

Part	Name	Scraping relevance
`a`	tag name	What kind of element it is. `soup.find('a')`
`class="product-link"`	attribute	Most common way you'll target elements
`href="/item/42"`	attribute	The actual data you often want (a link)
`data-id="42"`	custom data attribute	Sites hide useful IDs here
`Buy Now`	text content	What `.text` / `.get_text()` returns

Self-closing tags (no separate closing tag, no inner text): `<img>` , `<br>` , `<input>` , `<meta>` , `<hr>` . For these, the data you want is almost always in an **attribute**, not text — e.g. `img['src']` , not `img.text` (which will be empty).

---

3. Tags You'll See Constantly (and What They Usually Hold)

Tag	What it's usually for	What you extract
`div`, `span`	Generic containers — **no built-in meaning**. Identified only by class/id.	Whatever's nested inside; rarely the tag itself
`a`	Links	`.text` (link label) and `['href']` (URL)
`img`	Images	`['src']` or `['data-src']` (lazy-loaded images)
`p`	Paragraph text	`.text`
`h1` – `h6`	Headings	`.text`
`ul` / `ol` / `li`	Lists	Loop over `find_all('li')`
`table` / `thead` / `tbody` / `tr` / `th` / `td`	Tables	Row-by-row, cell-by-cell (see §10)
`form`, `input`, `select`, `option`	Forms	`['value']`, `['name']` — mostly relevant if you're submitting forms
`nav`, `header`, `footer`, `section`, `article`	"Semantic" layout tags	Useful landmarks to scope your search (e.g. "only search inside `<article>`")

Key insight: `div` and `span` are the most common tags on any real website *and* the least informative on their own. This is exactly why `class` and `id` exist — they're how developers (and you) tell generic boxes apart.

---

4. Attributes: The Real Scraping Targets

Attribute	What it means	Scraping use
`id="..."`	Meant to be **unique** on the page	Best-case selector — very reliable when present. `soup.find(id='price')`
`class="..."`	Meant to be **shared** by a group of similar elements	Your #1 tool for grabbing repeated items (product cards, list rows, etc.)
`href`	Destination URL (on `<a>`)	The link itself
`src`	Resource URL (on `<img>`, `<script>`)	Image/file URL
`data-*`	Custom attributes developers invent (`data-price`, `data-id`, `data-testid`)	Often the *cleanest* place to get raw values, sometimes cleaner than visible text
`name`	Identifies form fields	Only matters if scraping/submitting forms

Important nuance: `id` is *supposed* to be unique, but real-world websites break this rule constantly. Never assume — always sanity-check with DevTools how many elements share that id or class before trusting it.

A note on `class` with multiple values:

```
<div class="card featured highlighted">
```

This div has **three separate classes**. In CSS-selector terms `.card`, `.featured`, and `.highlighted` all match it individually. In BeautifulSoup, `class_="card"` matches even though there are other classes too — BS4 checks if the class is *in* the list, not an exact match.

---

5. Hierarchy: Parent, Child, Sibling, Descendant

This vocabulary matters because **the data you want and the attribute you can reliably find are often not on the same tag.**

```
<li class="review">
    <span class="stars">★★★★☆</span>
    <p class="comment">Great product, fast shipping.</p>
</li>
```

- `li.review` is the **parent** of both `span` and `p`.
- `span.stars` and `p.comment` are **siblings** of each other.
- Both are **children** of `li.review`.
- Both are also **descendants** of any tag further up the tree (like `<ul>` or `<div>` wrapping the whole list).

Why this matters in practice: Say you can easily find `p.comment` (unique class), but what you actually need is the star rating *next to* that specific comment. You don't re-search the whole page for `.stars` — you go up to the shared parent first, then back down:

```
comment_tag = soup.find('p', class_='comment')
container = comment_tag.parent          # go up to <li class="review">
stars = container.find('span', class_='stars').text   # go back down
```

This "go up, then back down" move is one of the most useful patterns in scraping. Related BeautifulSoup navigation tools:

Property/Method	Moves…
`.parent`	Up one level
`.parents`	Up through *all* ancestors (generator)
`.children`	Down one level (direct children only)
`.contents`	Down one level, as a list
`.descendants`	Down through everything nested inside
`.next_sibling` / `.previous_sibling`	Sideways to the next/previous node
`.find_next(tag)` / `.find_previous(tag)`	Sideways+forward/backward, skipping non-matches
`.find_parent(tag)`	Up, until a matching tag is found

6. Using Chrome DevTools Effectively

1. **Right-click → Inspect** on the exact data you want. This jumps the Elements panel straight to that node — much faster than manually browsing the tree.
2. **Hover over nodes in the Elements panel** — Chrome highlights the matching area on the page. Use this to confirm you've selected the right element, not a wrapper or a sibling.
3. `Ctrl+F` **inside the Elements panel** lets you search by tag name, class, or even a CSS selector — great for testing a selector *before* writing Python.
4. **Right-click a node → Copy → Copy selector** gives you a CSS selector Chrome thinks identifies that element. Treat this as a **starting point, not gospel** — Chrome's auto-generated selectors are often long, brittle, and over-specific (tied to exact position). You'll usually want to simplify it into something based on a meaningful class.
5. **View Page Source (Ctrl+U) vs. the Elements panel are NOT the same thing.**
   - "View Source" shows the raw HTML `requests.get()` will actually receive.
   - The Elements panel shows the **rendered DOM**, *after* JavaScript has run.
   - **This is the single most common scraping gotcha.** If data is visible on the page but missing from "View Source," it was likely injected by JavaScript — plain `requests` + `BeautifulSoup` won't see it. You'd need Selenium/Playwright, or better, check the Network tab for the API call that JavaScript used to fetch that data (see below).
6. **Network tab (XHR/Fetch filter):** Before reaching for Selenium, check if the data loads via a background API call. Many "JS-rendered" sites are actually just calling a clean JSON API behind the scenes — if you can find and call that URL directly with `requests`, it's far more reliable than scraping rendered HTML at all.

---

7. Finding the Pattern for Repeated Data (the actual hard skill)

Most real scraping targets are **lists of similar things**: search results, product cards, table rows, comments. The skill isn't finding *one* of them — it's finding the **one class or structure shared by all of them, and only them.**

Process: 1. Inspect *one* item (e.g., one product card). 2. Note its outer container tag and class, e.g. `<div class="product-card">`. 3. In the Elements panel, use `Ctrl+F` and search `.product-card` — check how many results come back. That number should match the number of items visible on the page. 4. If it matches → you've found your repeating unit. Loop over it:

```
cards = soup.find_all('div', class_='product-card')
for card in cards:
    name = card.find('h2', class_='title').text.strip()
    price = card.find('span', class_='price').text.strip()
    link = card.find('a')['href']
```

Why searching inside each `card` (not the whole `soup`) matters: if you did `soup.find_all('span', class_='price')` directly, you'd get every price on the page with no way to know which product each belongs to. Scoping your search to one container at a time keeps each item's data grouped correctly — this is the difference between messy, misaligned data and clean rows.

---

8. BeautifulSoup Methods Mapped to What You Just Learned

What you know about the HTML	BeautifulSoup code
Tag name only	`soup.find('div')` / `soup.find_all('div')`
Tag + single class	`soup.find('div', class_='price')`
Tag + id	`soup.find('div', id='main-price')`
Tag + custom attribute	`soup.find('div', attrs={'data-id': '42'})`
Multiple conditions at once	`soup.find('a', class_='btn', href=True)`
"Give me all matches"	`find_all(...)` → returns a **list**
"Give me just the first match"	`find(...)` → returns a **single tag** (or `None`)
You'd rather think in CSS syntax	`soup.select('div.price')` → list, like `find_all`
CSS syntax, first match only	`soup.select_one('div.price')` → like `find`

`find` / `find_all` **vs** `select` / `select_one`**:** functionally similar; `select` just accepts real CSS selector strings, which is often faster to write once you know CSS selector syntax (next section) and lets you express hierarchy (parent > child) in one line instead of chained `.find()` calls.

---

9. CSS Selector Cheat Sheet (for `.select()` / `.select_one()`)

Selector	Matches	Example
`tag`	All elements of that tag	`select('a')` — all links
`.class`	Elements with that class	`select('.price')`
`#id`	Element with that id	`select('#main-title')`
`tag.class`	Tag **and** class combined	`select('span.price')`
`parent child` (space)	Any descendant, at any depth	`select('div.card p')`
`parent > child`	**Direct** child only	`select('ul.list > li')`
`[attr]`	Has that attribute at all	`select('a[href]')`
`[attr="value"]`	Exact attribute match	`select('input[name="email"]')`
`[attr^="value"]`	Attribute **starts with**	`select('img[src^="https"]')`
`[attr$="value"]`	Attribute **ends with**	`select('a[href$=".pdf"]')`
`[attr*="value"]`	Attribute **contains**	`select('div[class*="product"]')`
`tag:nth-child(n)`	The nth child at that position	`select('tr:nth-child(2)')`
`A, B`	Matches A **or** B	`select('h1, h2')`
`.class1.class2`	Element has **both** classes (no space)	`select('.card.featured')`

When to prefer `[attr*="value"]`: Some sites generate dynamic/hashed class names (e.g. `class="css-x8f31k"`) that change on every deploy. If you notice a class looks random/unstable, look for a `data-*` attribute instead — those tend to be far more stable, and `[data-testid*="price"]` can be a lifesaver.

---

10. Getting Data Out Once You've Found the Tag

You want…	Code	Notes
Visible text	`tag.text` or `tag.get_text()`	`.get_text(strip=True)` removes leading/trailing whitespace/newlines — use this by default
Text with a custom separator (multiple nested text pieces)	`tag.get_text(separator=' ', strip=True)`	Prevents words from different lines getting jammed together
An attribute value	`tag['href']` or `tag.get('href')`	Use `.get()` if the attribute might be missing — avoids a `KeyError` crash
Only the tag's own direct text (not children's)	`tag.string`	Returns `None` if the tag contains more than one child element — stricter than `.text`
A list of matches	`soup.find_all(...)` → loop with `for`	Always returns a list, even if empty — never crashes, but check `len()` before assuming data exists

---

11. Common Real Structures, Solved

A table:

```
table = soup.find('table')
rows = table.find_all('tr')
for row in rows:
    cells = row.find_all(['td', 'th'])
    values = [cell.get_text(strip=True) for cell in cells]
```

A simple list:

```
items = soup.select('ul.tags > li')
tags = [item.get_text(strip=True) for item in items]
```

A grid of cards (the most common e-commerce/listing pattern):

```
cards = soup.select('div.product-card')
data = []
for card in cards:
    data.append({
        'name': card.select_one('.title').get_text(strip=True),
        'price': card.select_one('.price').get_text(strip=True),
        'url': card.select_one('a')['href'],
    })
```

Lazy-loaded images (real image URL often isn't in `src` until you scroll — it's frequently stashed in a `data-*` attribute instead):

```
img = card.find('img')
real_url = img.get('data-src') or img.get('src')
```

Pagination links (look for the "next page" pattern):

```
next_link = soup.select_one('a.next, a[rel="next"]')
next_url = next_link['href'] if next_link else None
```

---

12. Common Pitfalls (Save Yourself the Debugging Time)

Symptom	Likely cause	Fix
`soup.find(...)` returns `None` unexpectedly	Data is JS-rendered, not in raw HTML	Check "View Source" vs Elements panel (§6). Look at Network tab for a JSON API instead.
`.text` gives jumbled/missing spacing	Nested tags glued together	Use `get_text(separator=' ', strip=True)`
`find_all` grabs way more elements than expected	Class name is reused elsewhere on the page for unrelated elements	Scope your search inside a more specific parent container first
Selector works in DevTools but not in Python	Page structure differs between what JS renders and what `requests` actually downloads	Same as JS-rendered issue above — compare raw vs rendered HTML
`KeyError` on `tag['href']`	That particular tag doesn't have the attribute	Use `tag.get('href')` instead, which returns `None` instead of crashing
Class name looks like `css-1a2b3c` and changes every time you reload	Auto-generated/hashed class from a CSS-in-JS framework	Don't rely on it — target a `data-*` attribute or structural position instead
Works for the first item, breaks on later ones	You assumed all repeated cards have identical inner structure — some have missing fields (e.g. no discount badge)	Use `.get()` / check `if tag:` before pulling data out of optional sub-elements

---

13. Quick Reference — The Whole Cheat Sheet in One Table

HTML Pattern	What it usually means	BeautifulSoup / CSS
`<div class="x">` repeated many times	A list of similar items	`soup.find_all('div', class_='x')` or `soup.select('div.x')`
`<div id="x">`	A unique element	`soup.find(id='x')` or `soup.select_one('#x')`
`<a href="...">`	A link	`.find('a')['href']`
`<img src="...">` or `data-src`	An image	`.get('src')` or `.get('data-src')`
`<table><tr><td>`	Tabular data	Loop `find_all('tr')` → `find_all(['td','th'])`
`<ul><li>`	A simple list	`soup.select('ul > li')`
Text inside any tag	Visible content	`.get_text(strip=True)`
`data-*` attribute	Custom/stable data hook	`.get('data-something')`
Element only visible after scrolling/interacting	JS-rendered	Check Network tab for underlying API, or use Selenium/Playwright

---

The habit to build

Every time you inspect a page, ask three questions in this order: 1. **What's the smallest container that wraps exactly the one item I want (or one repeat unit, if it's a list)?** 2. **What class, id, or attribute uniquely (or reliably) identifies that container?** 2. **From there, is the data I need on this exact tag, or do I need to move to a parent/child/sibling to get it?**

Answer those three, and the BeautifulSoup code basically writes itself.